

面向对象设计原则在电商前端项目中的应用实践

一、里氏替换原则（Liskov Substitution Principle, LSP）

定义：子类必须能够替换其父类，且程序行为不变。

本项目应用举例：

- 在SKU选择器（`XtxSku`）组件中，我们定义了统一的商品规格选择接口。不同商品类型（如普通商品、预售商品、秒杀商品）都可以通过继承同一个基础SKU组件来实现自己的选择逻辑，但在父组件调用时，无需关心具体类型，只需调用统一接口即可。
- 例如：`BaseSku` → `NormalSku` / `PreSaleSku`，在购物车或详情页中，直接使用 `BaseSku` 类型引用，运行时替换为子类实例，行为仍然正确。

```
// 伪代码
class BaseSku {
  selectSpec(spec) { /* 通用逻辑 */ }
}
class NormalSku extends BaseSku { /* 无额外重写 */ }
class PreSaleSku extends BaseSku {
  selectSpec(spec) {
    // 增加预售逻辑，但不破坏原有行为
  }
}
```

二、单一职责原则（Single Responsibility Principle, SRP）

定义：一个类/模块只应有一个引起它变化的原因。

本项目应用举例：

- `CartStore`（购物车状态）只负责购物车数据的存储、持久化、增删改查，不参与UI渲染或网络请求细节。

- **Axios** 封装模块只负责HTTP请求与响应拦截，不处理业务逻辑（如401后的跳转交给路由模块）。
- **Layout** 组件只负责布局结构，不包含任何数据获取逻辑。

好处：当购物车存储方式从 `localStorage` 改为后端存储时，只需修改 `CartStore`，不影响其他模块。

三、开闭原则（Open-Closed Principle, OCP）

定义：对扩展开放，对修改关闭。

本项目应用举例：

- 在**路由权限控制**中，我们通过“扩展点”方式支持不同角色（买家、卖家、管理员）的页面访问控制。新增角色时，只需在权限配置数组中添加新条目，而不需要修改原有的路由守卫逻辑。

```
// roles.js
export const roleRoutes = {
  buyer: ['/home', '/cart', '/order'],
  seller: ['/seller/product'],
  admin: ['/admin/user']
};
// 新增一个 guest 角色，只需扩展配置，不修改核心守卫函数
```

- 在**表单校验**模块中，通过策略模式支持不同表单的校验规则扩展，而无需修改校验引擎。

四、迪米特法则（Law of Demeter, LoD）

定义：一个对象应尽可能少地了解其他对象的内部结构。

本项目应用举例：

- 在**结算页**中，`checkInfo` 聚合了地址、商品、优惠等信息，结算页组件只与 `checkInfo` 交互，而不直接调用地址Store、购物车Store等多个内部模块。
- **组件之间通过事件/状态通信，避免直接引用对方内部方法**。例如：商品详情页修改购物车数量后，只触发一个 `cartUpdated` 事件，购物车列表页监听该事件后自行刷新，而不是详情页直接调用购物车页的刷新方法。

五、依赖倒转原则 (Dependency Inversion Principle, DIP)

定义：高层模块不应依赖低层模块，二者都应依赖抽象；抽象不应依赖细节，细节应依赖抽象。

本项目应用举例：

- **HTTP封装**：所有业务模块（商品、购物车、订单）都依赖 `axiosInstance` 抽象，而不是直接依赖 `axios` 或 `fetch`。未来更换网络库时，只需修改 `axiosInstance` 内部实现，业务代码零修改。
- **状态管理**：组件依赖 `useCartStore` 等Pinia抽象，而不是直接操作 `localStorage` 或 `sessionStorage`。

```
// 业务模块中
import { useCartStore } from '@/stores/cart';
const cartStore = useCartStore();
// 而不是：localStorage.setItem('cart', ...)
```

六、合成复用原则 (Composite Reuse Principle, CRP)

定义：优先使用对象组合（合成/聚合），而不是类继承来实现复用。

本项目应用举例：

- **懒加载指令**（`v-img-lazy`）作为可复用的组合功能，被注入到任何图片组件中，而不是让所有图片组件继承某个基类。
- **公共Layout布局**：头部、导航、侧边栏、底部等通过插槽（`slot`）组合，而不是通过多层继承实现不同页面布局。
- **SKU选择器**：内部包含“规格按钮组”、“数量选择器”、“价格展示”等多个小组件的组合，而不是通过继承方式扩展功能。

```
<!-- 组合示例 -->
<template>
  <Layout>
    <template #header><Header /></template>
    <template #main><ProductDetail /></template>
    <template #footer><Footer /></template>
  </Layout>
</template>
```

总结对照表

原则	项目实践模块	效果
里氏替换	SKU选择器、商品类型扩展	多态替换安全
单一职责	CartStore、Axios封装、Layout	模块清晰，易维护
开闭原则	权限路由、表单校验策略	扩展角色/规则无需改核心
迪米特法则	结算页聚合checkInfo、组件事件通信	低耦合，高内聚
依赖倒转	HTTP抽象、Pinia状态抽象	底层替换不影响业务
合成复用	懒加载指令、Layout插槽、SKU组合	灵活组合，避免继承滥用